DATA Structure and Algorithms

(BCO 002B)

UNIT 2

Prepared By:
Mukesh Kumar
Assistant Professor
Department of Computer Science & Engineering
JECRC University

Syllabus

BCO002B	DATA STURCTURES AND ALGORITHM	4-0-0 [4]
UNIT 1	Introduction: Notions of data type, abstract data type and data structures. Importance of algorithms and data structures in programming. Notion of Complexity covering time complexity, space complexity, Worst case complexity & Average case complexity. BigOh Notation, Omega notation,Theta notation. Examples of simple algorithms and illustration of their complexity. Sorting- Bubble sort, selection sort, insertion sort, Quick sort; Heap sort; Merge sort; Analysis of the sorting methods. Selecting the top k elements. Lower bound on sorting.	
UNIT 2	Stack ADT, Infix Notation, Prefix Notation and Postfix Notation. Evaluation of Postfix Expression, conversion of Infix to Prefix and Postfix Iteration and Recursion- Problem solving using iteration and recursion with examples such as binary search, Fibonacci numbers, and Hanoi towers. Tradeoffs between iteration and recursion.	
UNIT 3	List ADT. Implementation of lists using arrays and pointers. Stack ADT. Queue ADT. Implementation of stacks and queues. Dictionaries, Hash tables: open tables and closed tables. Searching technique- Binary search and linear search, link list- single link list, double link list, Insertion and deletion in link list.	
UNIT 4	Binary Trees- Definition and traversals: preorder, post order, in order. Common types and properties of binary trees. Binary search trees: insertion and deletion in binary search tree worst case analysis and average case analysis. AVL trees. Priority Queues -Binary heaps: insert and delete min operations and analysis.	
UNIT 5	Graph: Basic definitions, Directed Graphs- Data structures for graph representation. Shortest path algorithms: Dijkstra (greedy algorithm) and Operations on graph, Worshall's algorithm , Depth first search and Breadth-first search. Directed acyclic graphs. Undirected Graphs, Minimal spanning trees and algorithms (Prims and Kruskal) and implementation. Application to the travelling salesman problem.	

Course Outcome	Program Outcome												Program Specifice Outcome		
	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2	PSO3
CO1	H												M		
CO2			H		M								M		
CO3		H							M			L		H	
CO4	H	M											L		L
CO5		M		H											L

Course Objective

CO1:Show the understanding of asymptotic analysis of algorithm
CO2: Understand the concepts of Stacks and Queues with their application
CO3: Understand with utilization concepts of Linked List with their application
CO4:Use comprehensive knowledge Trees in problem solving.
CO5:Use comprehensive knowledge of Graph.

Books & References

Text Books:

1. Data Structures and Algorithms by Alfred V. Aho, Jeffrey D. Ullman and John E. Hopcroft , Addison-Wesley Series (1983)

Reference Books:

T.H. Cormen, C.E. Leiserson, and R.L. Rivest. Introduction to Algorithms.The MIT Press and McGraw-Hill Book Company, Cambridge, Massacusetts, 1990 (Available in Indian Edition).
Steven S. Skiena. The Algorithm Design Manual.Springer, Second Edition, 2008.
Data Structures and Algorithm Analysis in Java (3rd Edition) by Mark Allen Weiss, Addison Wesley(2011)

Stack and its operations

A Stack is a linear data structure in which an element may be inserted or deleted only at one end, called the TOP of the Stack.

For example:-

-- a stack of dishes

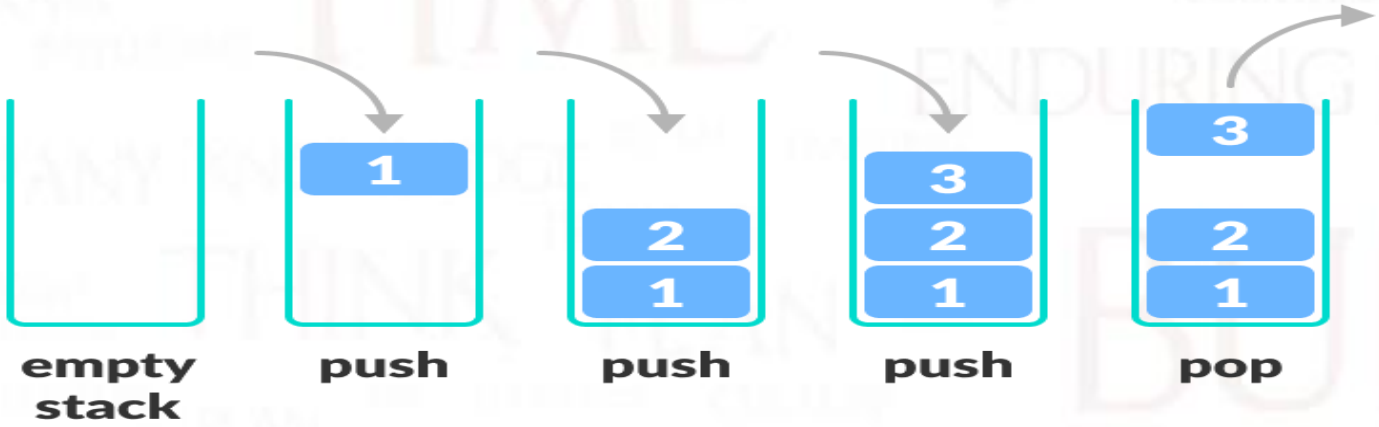
-- a stack of folded towels

Stack is based on LIFO(Last in first out).

Stack and its operations

LIFO Principle of Stack

In programming terms, putting an item on top of the stack is called push and removing an item is called pop.



Basic Operations on Stack

Push: Adds an element to the top of the stack.
Pop: Removes the top element from the stack.
Peek: Returns the top element without removing it.
IsEmpty: Checks if the stack is empty.
IsFull: Checks if the stack is full (in case of fixed-size arrays).

Representation Of stack

Representation Of stack:
(i) Array Representation
(ii) Linked List Representation

A stack can be implemented by means of Array and Linked List. Stack can either be a fixed size or array representation Stack may have a sense of dynamic resizing i.e linked list representation.

Array Representation of stack

Working of Stack Data Structure

The operations work as follows:

A variable called TOP is used to keep track of the top element in the stack.

When initializing the stack, we set its value to -1 so that we can check if the stack is empty by comparing TOP == -1.

On pushing an element, we increase the value of TOP and place the new element in the position pointed to by TOP.

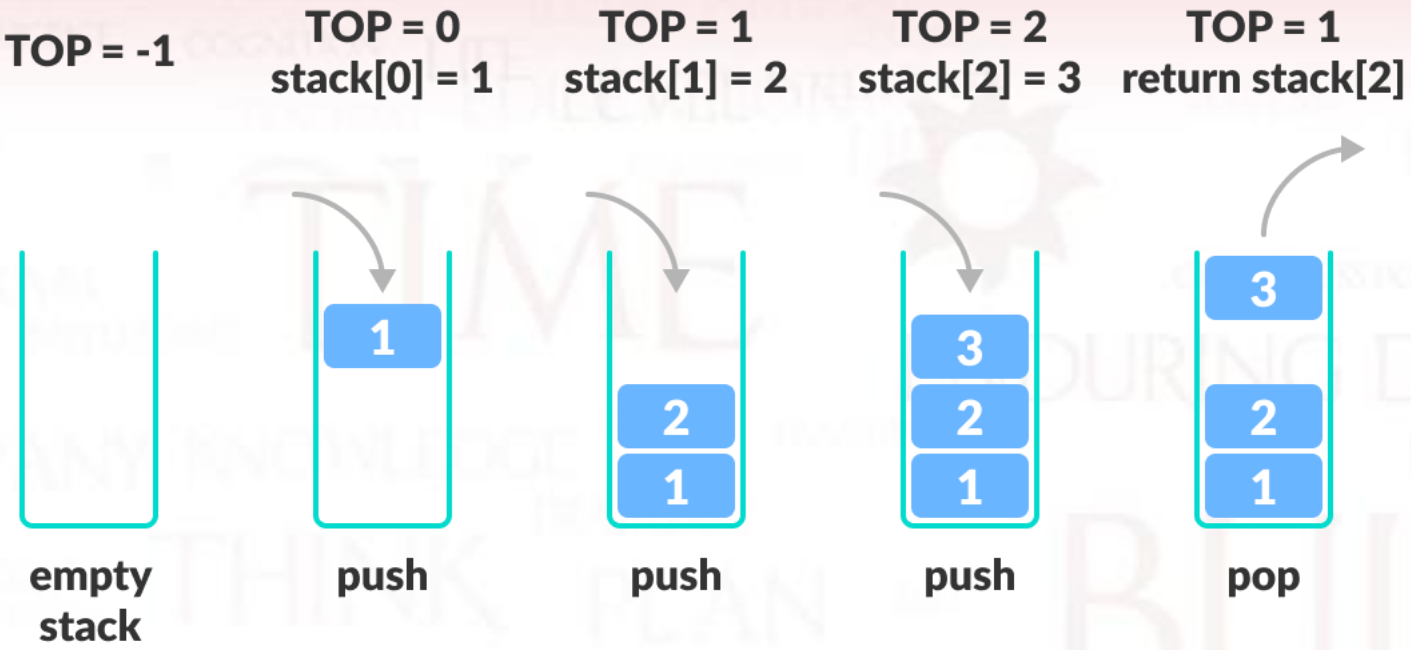
On popping an element, we return the element pointed to by TOP and reduce its value.

Before pushing, we check if the stack is already full then overflow occur

Before popping, we check if the stack is already empty, if empty then underflow occur

Implementing Stacks: Array

- Advantages
  - best performance
- Disadvantage
  - fixed size
- Basic implementation
  - initially empty array
  - field to record where the next data gets placed into
  - if array is full, push() returns false
    - otherwise adds it into the correct spot
  - if array is empty, pop() returns null
    - otherwise removes the next item in the stack


TOP = -1
TOP = 0
stack[0] = 1
TOP = 1
stack[1] = 2
TOP = 2
stack[2] = 3
TOP = 1
return stack[2]
1
2
1
3
2
1
3
2
1
empty stack
push
push
push
pop

Push Operation

The process of putting a new data element onto stack is known as a Push Operation. Push operation involves a series of steps −

Step 1 − Checks if the stack is full.

Step 2 − If the stack is full, produces an error and exit.

Step 3 − If the stack is not full, increments **top** to point next empty space.

Step 4 − Adds data element to the stack location, where top is pointing.

Step 5 − Returns success.

The steps involved in the PUSH operation is given below:

Before inserting an element in a stack, we check whether the stack is full.

If we try to insert the element in a stack, and the stack is full, then the ***overflow*** condition occurs.

When we initialize a stack, we set the value of top as -1 to check that the stack is empty.

When the new element is pushed in a stack, first, the value of the top gets incremented, i.e., **top=top+1,** and the element will be placed at the new position of the **top**.

The elements will be inserted until we reach the ***max*** size of the stack.

Algorithms for PUSH operation

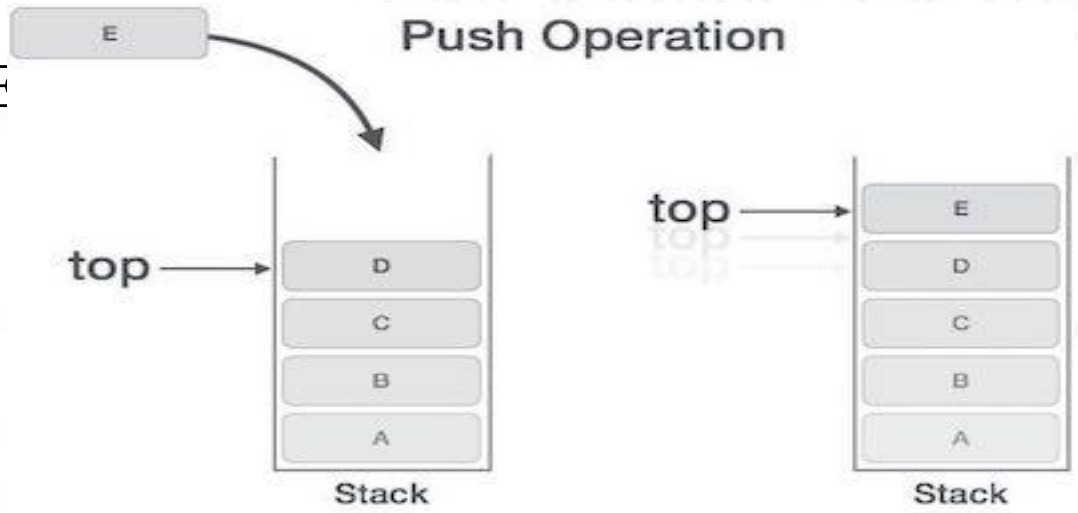
Push Operation in Stack Data Structure:

Insert element in stack. If the stack is full, then it is said to be an Overflow condition.

Algorithm for Insertion:-

PUSH(STACK, TOP, MAXSTK, ITEM)

1. If TOP=MAXSTK, then print OVERFLOW and Exit.
2. Set TOP=TOP+1
3. Set STACK[TOP]= ITE
4. Return



Accessing the content while removing it from the stack, is known as a Pop Operation. In an array implementation of pop() operation, the data element is not actually removed, instead **top** is decremented to a lower position in the stack to point to the next value. But in linked-list implementation, pop() actually removes data element and deallocates memory space.

The steps involved in the POP operation is given below:

Before deleting the element from the stack, we check whether the stack is empty.

If we try to delete the element from the empty stack, then the ***underflow*** condition occurs.

If the stack is not empty, we first access the element which is pointed by the ***top***

Once the pop operation is performed, the top is decremented by 1, i.e., **top=top-1**.

The steps involved in the POP operation is given below:

Before deleting the element from the stack, we check whether the stack is empty.

If we try to delete the element from the empty stack, then the ***underflow*** condition occurs.

If the stack is not empty, we first access the element which is pointed by the ***top***

Once the pop operation is performed, the top is decremented by 1, i.e., **top=top-1**.

A Pop operation may involve the following steps −

Step 1 − Checks if the stack is empty.

Step 2 − If the stack is empty, produces an error and exit.

Step 3 − If the stack is not empty, accesses the data element at which **top** is pointing.

Step 4 − Decreases the value of top by 1.

Step 5 − Returns success.

Algorithms for POP operation

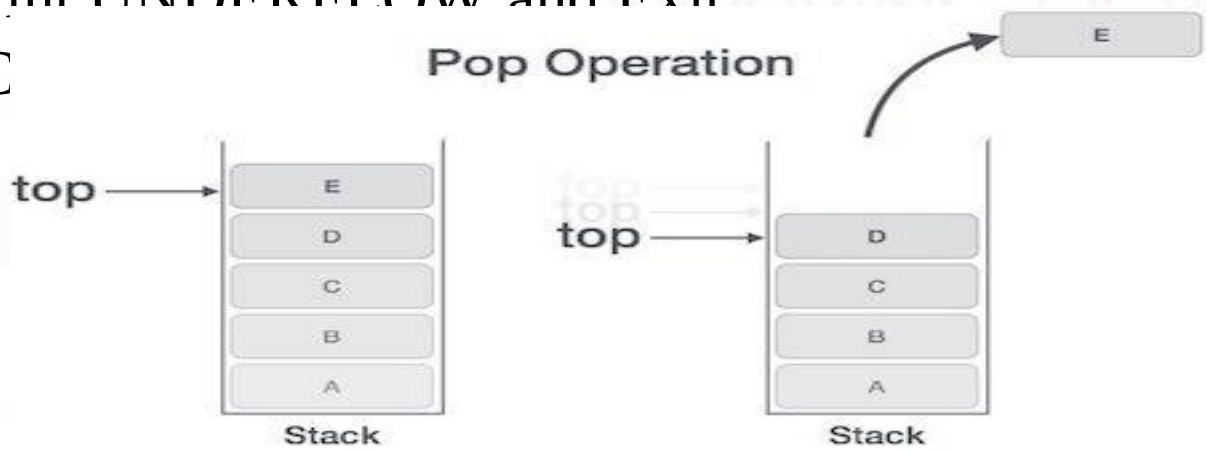
Pop Operation in Stack Data Structure:

Removes an item from the stack. The items are popped in the reversed order in which they are pushed. If the stack is empty, then it is said to be an Underflow condition.

Algorithm for Insertion:-

POP(STACK, TOP, ITEM)

1. If TOP=0, then print UNDERFLOW and Exit
2. Set ITEM= STAC
3. Set TOP=TOP-1
4. Return



Applications of Stack Data Structures

Recursion
Expression Evaluation
Undo/Redo Operations
Browser History
Function Calls
Stacks can be used to check parenthesis matching in an expression.
Stacks can be used for Conversion from one form of expression to another.

Stack Applications

- Stacks are a very common data structure
  - compilers
    - parsing data between delimiters (brackets)
  - operating systems
    - program stack
  - virtual machines
    - manipulating numbers
      - pop 2 numbers off stack, do work (such as add)
      - push result back on stack and repeat
  - artificial intelligence
    - finding a path

Notations and its types

***Definition*:-** The way to write arithmetic expression is known as a **notation**.

An arithmetic expression can be written in three different but equivalent notations, i.e., without changing the essence or output of an expression.

These notations are −

- **Infix Notation**
- **Prefix (Polish) Notation**
- **Postfix (Reverse-Polish) Notation**

These notations are named as how they use operator in expression.

***Infix notation*:-** The operator symbol is placed between its two operands.

For example:- A+B, E*G, (A+B)*C etc.

***Prefix notation*:-** The operator symbol is placed before its two operands.

For example:- +AB, *EG, *+ABC

-- it is also called Polish notation.

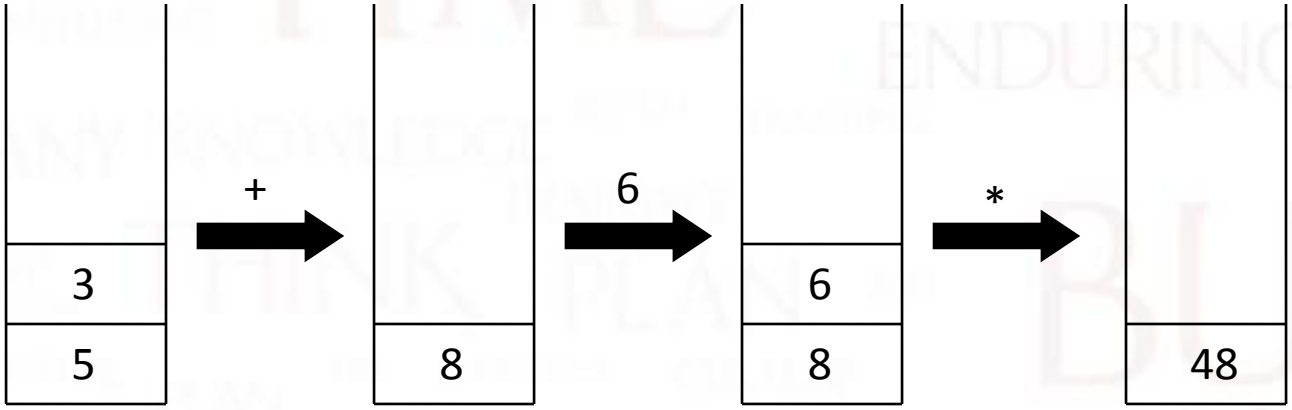
***Postfix notation*:-** The operator symbol is placed after its two operands.

For example:- AB+, *EG, AB+C*

-- it is also called Reverse Polish notation.

Reverse Polish Notation

- Way of inputting numbers to a calculator
  - (5 + 3) * 6 becomes 5 3 + 6 *
  - 5 + 3 * 6 becomes 5 3 6 * +
- We can use a stack to implement this
  - consider 5 3 + 6 *



  - try doing 5 3 6 * +

Sr.No.	Infix Notation	Prefix Notation	Postfix Notation
1	a + b	+ a b	a b +
2	(a + b) ∗ c	∗ + a b c	a b + c ∗
3	a ∗ (b + c)	∗ a + b c	a b c + ∗
4	a / b + c / d	+ / a b / c d	a b / c d / +
5	(a + b) ∗ (c + d)	∗ + a b + c d	a b + c d + ∗
6	((a + b) ∗ c) - d	- ∗ + a b c d	a b + c ∗ d -

In **a + b*c**, the expression part **b*c** will be evaluated first, with multiplication as precedence over addition. We here use parenthesis for **a + b** to be evaluated first, like **(a + b)*c**.

Postfix Evaluation Algorithm

We shall now look at the algorithm on how to evaluate postfix notation −

Step 1 − scan the expression from left to right

Step 2 − if it is an operand push it to stack

Step 3 − if it is an operator pull operand from stack and perform operation

Step 4 − store the output of step 3, back to stack

Step 5 − scan the expression until all operands are consumed

Step 6 − pop the stack and perform operation

Algorithm to convert postfix to infix:

Step 1: Scan the given postfix expression from left to right character by character.

Step 2: If the character is an operand, push it into the stack.

Step 1: But if the character is an operator, pop the top two values from stack.

Concatenate this operator with these two values (2nd top value+operator+1st top value) to get a new string.

Now push this resulting string back into the stack.

Repeat this process untill the end of postfix expression. Now the value in the stack is the infix expression.

Input String	Postfix Expression	Stack (Infix)
ab*c+	b*c+	a
ab*c+	*c+	ab
ab*c+	c+	(a*b)
ab*c+	+	(a*b)c
ab*c+		((a*b)+c)

Example to Convert Postfix to Infix

Postfix Expression : abc-+de-+

Token	Stack	Action
a	a	push a in stack
b	a, b	push b in stack
c	a, b, c	push c in stack
–	a , b – c	pop b and c from stack and put – in between and push into stack
+	a + b – c	pop a and b-c from stack and put + in between and push into stack
d	a + b – c, d	push d in stack
e	a + b – c, d , e	push e in stack
–	a + b – c, d – e	pop d and e from stack and put – in between and push into stack
+	a + b – c + d – e	pop a + b – c and d – e from stack and put + in between and push into stack

Evaluation of Postfix Expression

Suppose P is an arithmetic expression written in postfix notation. The following algorithm, which uses a STACK to hold operands, evaluates P.

Step 1 :- Add “)” at the end of P and scan P from left to right
Step 2:- if it is an operand push it to stack
Step 3:-if it is an operator pull two operands from stack and perform operation
Step 4:- store the output of step 3, back to stack
Step 5:-scan the expression until all operands are consumed
Step 6:-pop the stack and perform operation

Conversion from infix to postfix expression using stack

Suppose Q is an arithmetic expression written in infix notation, the algorithm finds the equivalent postfix expression P.

POLISH(Q,P)

1.Push “(“ onto STACK and add “)” to the end of Q.

2.Scan Q from left to right until Stack is empty.

3.If operand is encountered, add it to P

4.If “(“ is encountered , push it onto STACK.

5.If an operator is encountered, then

(a) Repeatedly pop from STACK and add to P each operator which has the same as or higher precedence than operator.

(b) Add operator to STACK.

6. If a “)” is encountered, then:

(a) Repeatedly pop from STACK and add to P each operator until “(“ is encountered.

(b) Remove “(“.

7. Exit

For example:- Consider the following infix expression Q

Q: ((A+B)*D)

Solution:- Push "(" into Stack and add ")" at the end of Q.

Symbol	STACK	Expression P
(	((	
(	(((	
A	(((	A
+	(((+	A
B	(((+	AB
)	((	AB+
*	((*	AB+
D	((*	AB+D
)	(	AB+D*
)	-	AB+D*

Recursion

***Definition*:-** A function is said to be recursively defined if the function definition refers to itself.

A programming technique in which a function calls itself.

It must have following two properties:-

❖There must be certain criteria, called *base values*, for which the function does not refers to itself.

❖Each time the function does refer to itself, the argument of the function must be closer to a base value.

A function with these two properties is also said to be well defined.

Recursion

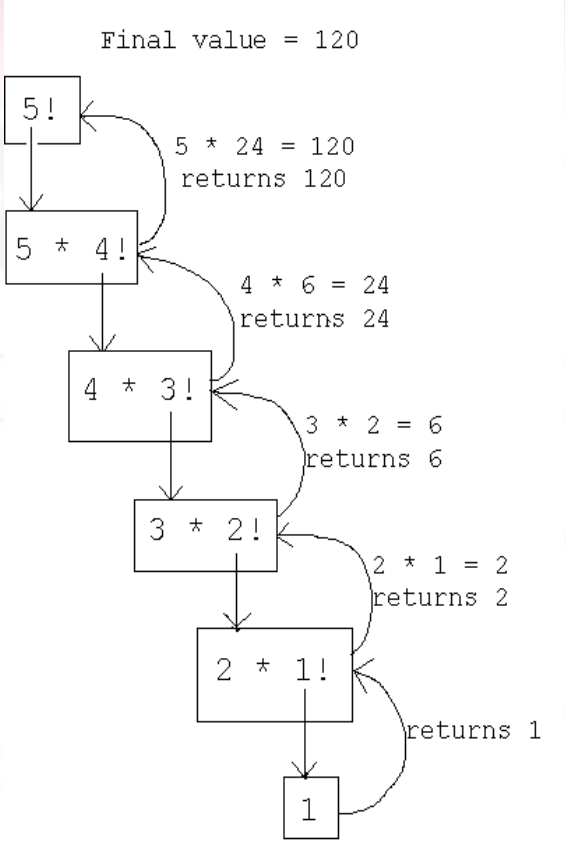
- Recursion is one way to decompose a task into smaller subtasks. At least one of the subtasks is a smaller example of the same task.
- The smallest example of the same task has a non-recursive solution.

Example: The factorial function

```
n! = n * (n-1)! and 1! = 1
```

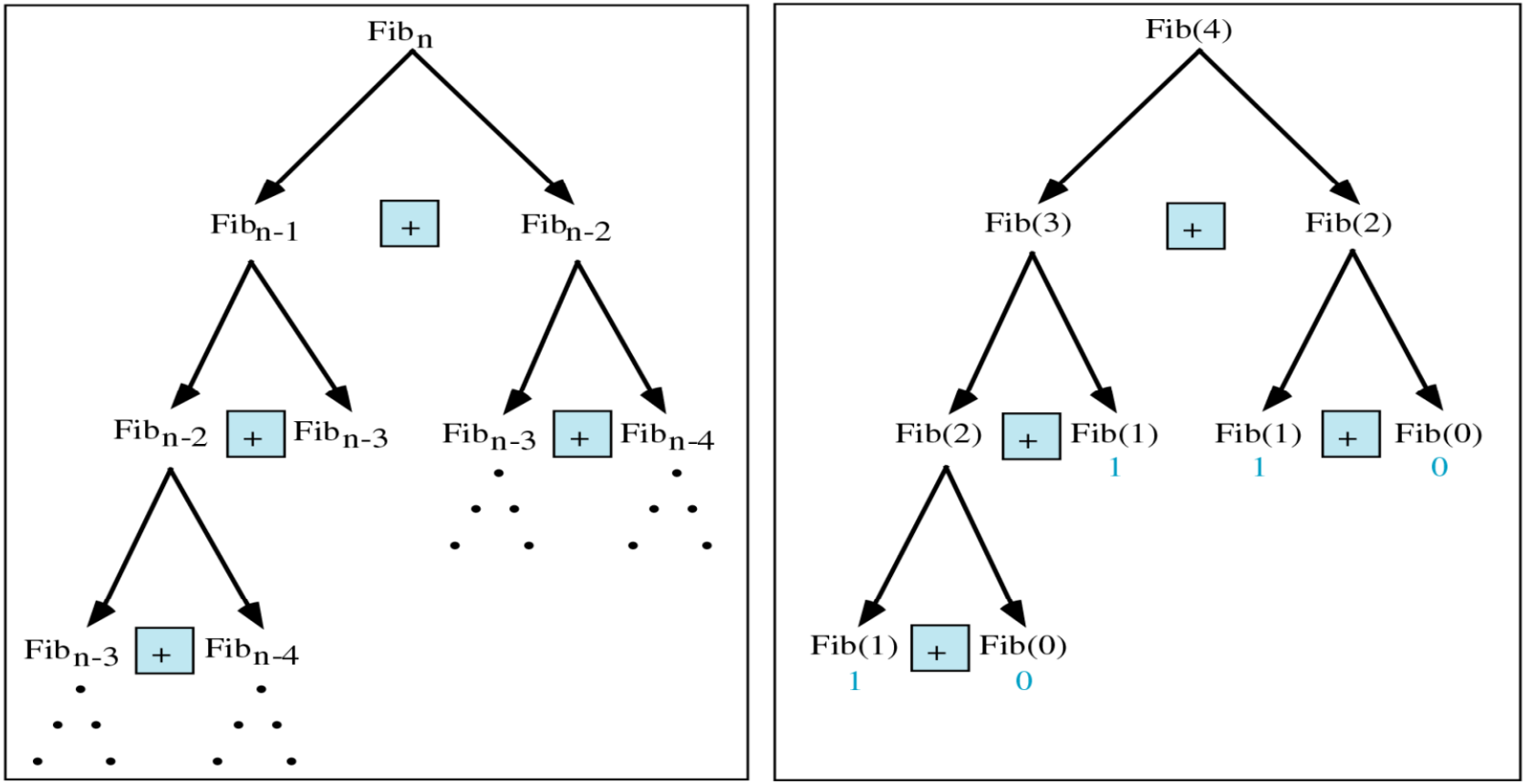
For example: recursive implementation of factorial function.

```
int Factorial(int n)
{
 if (n==0)  // base value
   return 1;
 else
   return n * Factorial(n-1);
}
```



Fibonacci numbers

```
//Calculate Fibonacci numbers using recursive function.
//A very inefficient way, but illustrates recursion well
int fib(int number)
{
        if (number == 0) return 0;
        if (number == 1) return 1;
        return (fib(number-1) + fib(number-2));
}

int main(){         // driver function
        int inp_number;
        cout << "Please enter an integer: ";
        cin >> inp_number;
        cout << "The Fibonacci number for "<< inp_number
                << " is "<< fib(inp_number)<<endl;
  return 0;
}
```


Fib$_n$
Fib$_{n-1}$
+
Fib$_{n-2}$
Fib$_{n-2}$
+
Fib$_{n-3}$
Fib$_{n-3}$
+
Fib$_{n-4}$
Fib$_{n-3}$
+
Fib$_{n-4}$
Fib(4)
Fib(3)
+
Fib(2)
Fib(2)
+
Fib(1)
1
Fib(1)
1
+
Fib(0)
0
Fib(1)
1
+
Fib(0)
0

(a) Fib(n) (b) Fib(4)

Binary Search

Given a sorted array arr[] of n elements, write a function to search a given element x in arr[].

A simple approach is to do linear search.The time complexity of above algorithm is O(n). Another approach to perform the same task is using Binary Search.

Binary Search: Search a sorted array by repeatedly dividing the search interval in half. Begin with an interval covering the whole array. If the value of the search key is less than the item in the middle of the interval, narrow the interval to the lower half. Otherwise narrow it to the upper half. Repeatedly check until the value is found or the interval is empty.

Example :

Binary Search

	0	1	2	3	4	5	6	7	8	9
Search 23	2	5	8	12	16	23	38	56	72	91
23 > 16 take 2nd half	L=0	1	2	3	M=4	5	6	7	8	H=9
	2	5	8	12	16	23	38	56	72	91
23 > 56 take 1st half	0	1	2	3	4	L=5	6	M=7	8	H=9
	2	5	8	12	16	23	38	56	72	91
Found 23, Return 5	0	1	2	3	4	L=5, M=5	H=6	7	8	9
	2	5	8	12	16	23	38	56	72	91

We basically ignore half of the elements just after one comparison.
Compare x with the middle element.
If x matches with middle element, we return the mid index.
Else If x is greater than the mid element, then x can only lie in right half subarray after the mid element. So we recur for right half.
Else (x is smaller) recur for the left half.

Binary Search

Binary search

Compare the search element with the middle element of the array

If not equal, then apply binary search to half of the array (if not empty) where the search element would be.

Binary Search with Recursion

```
// Searches an ordered array of integers using recursion
int bsearchr(const int data[], // input: array
            int first,        // input: lower bound
            int last,         // input: upper bound
            int value         // input: value to find
        )// output: index if found,  otherwise return –1

{  //cout << "bsearch(data, "<<first<< ", last "<< ", "<<value << "); "<<endl;
   int middle = (first + last) / 2;
   if (data[middle] == value)
      return middle;
   else if (first >= last)
      return -1;
   else if (value < data[middle])
      return bsearchr(data, first, middle-1, value);
   else
      return bsearchr(data, middle+1, last, value);
}
```

Binary Search

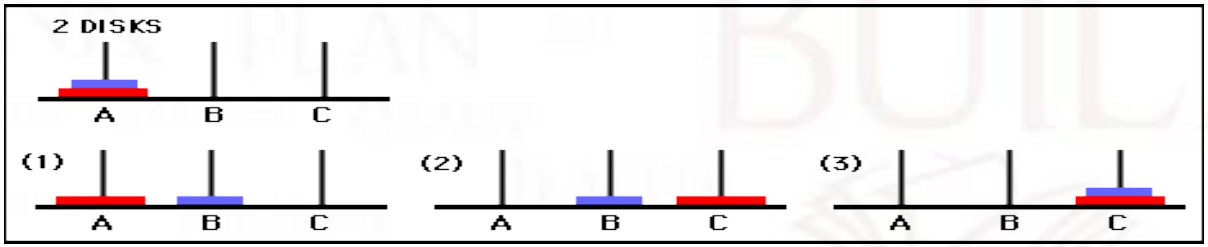
```
int main() {
            const int array_size = 8;
    int list[array_size]={1, 2, 3, 5, 7, 10, 14, 17};
    int search_value;

    cout << "Enter search value: ";
    cin >> search_value;
    cout << bsearchr(list,0,array_size-1,search_value)
        << endl;

    return 0;
}
```

Towers of Hanoi

Suppose three pegs, labeled A, B and C, are given and suppose on peg A there are finite number n disks with decreasing size.
The objective of the game is to move the disks from peg A to peg C using peg b as an auxiliary.
The rules of the game are as follows:-

❖ only one disk may be moved at a time. Specifically, only the top disk on any peg may be moved to any other peg.

❖At no time can a larger disk be placed on a smaller disk.



General Solution

The technique of recursion is used to solve tower of Hanoi problem.

For n==1

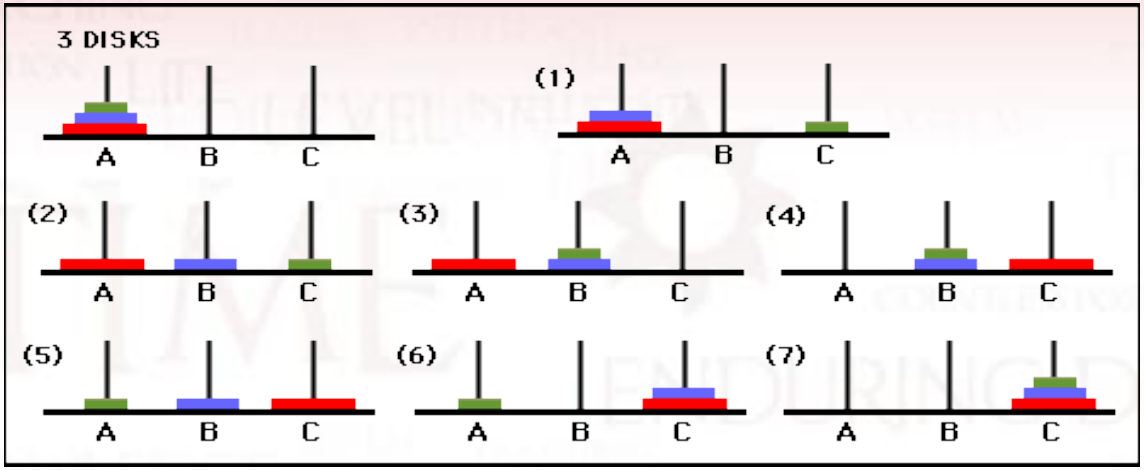
1. move disks from peg A to peg C

For n>1 disks may be reduced to the following sub problems:

1. Move the top n-1 disks from peg A to peg B.
2. Move the top disk from peg A to peg C.
3. Move the top n-1 disks from peg B to peg C.

In general, this recursive solution requires $f(n)= 2^n-1$ moves for n disks.

For example:- for n=3, the solution is given below



There are following moves:-

1.A to C
2. A to B
3. C to B
4. A to C
5. B to A
6. B to C
7. A to C

Implementation Towers of Hanoi

```
void hanoi(int from, int to, int num)
{
    int temp = 6 - from - to; //find the temporary
                              //storage column
    if (num == 1){
        cout << "move disc 1 from " << from
             << " to " << to << endl;
}
    else {
        hanoi(from, temp, num - 1);
        cout << "move disc " << num << " from " << from
             << " to " << to << endl;
        hanoi(temp, to, num - 1);
    }
}
```

```
int main() {
    int num_disc;  //number of discs

    cout << "Please enter a positive number (0 to quit)";
    cin >> num_disc;

    while (num_disc > 0){
        hanoi(1, 3, num_disc);
        cout << "Please enter a positive number ";
        cin >> num_disc;
    }
    return 0;
}
```

Factorial

Factorial of a number is the number you get by multiplying all the numbers up to that number including the number itself. The program for factorial does not use a programming technique called a recursion. a recursion happens when a function calls itself until the problem is solved.

Problem Definition

The program requires user input – a positive integer value and computes the factorial of than number.

For example,

Suppose you entered number 5 , then the factorial of this number would be

1 x 2 x 3 x 4 x 5 = 120

In mathematical terms, if n is a positive integer value, the factorial of n is denoted by n! You can go through the flowchart to understand the logic of factorial applied in this program.

Factorial Definition

- In some problems, it may be natural to define the problem in terms of the problem itself.
- Recursion is useful for problems that can be represented by a simpler version of the same problem.
- Example: the factorial function

```
6! = 6 * 5 * 4 * 3 * 2 * 1
```

We could write:

```
6! = 6 * 5!
```

```
Factorial(5)
↑
return 5 * Factorial(4) = 120
↑
return 4 * Factorial(3) = 24
↑
return 3 * Factorial(2) = 6
↑
return 2 * Factorial(1) = 2
↑
1
```

Recursive Function to calculate factorial

```
int factorial(int n) {
  //base case
  if(n == 0) {
    return 1;
  } else {
    return n * factorial(n-1);
  }
}
int main()
{
    int n=5,  res;
    res= factorial(n);
    printf("%d",res);
}
```

Recursion v/s Iteration

in terms of the base case gives us an idea of the time complexity of recursive equations study the different aspects of both recursive and iterative approaches.

1. **Time Complexity**

The time complexity of the method may vary depending on whether the algorithm is implemented using recursion or iteration.

•**Recursion**: Recursion involves calling the same function again, and hence, has a very small length of code. However, as we saw in the analysis, the time complexity of recursion can get to be exponential when there are a considerable number of recursive calls.

Iteration: Iteration is the repetition of a block of code. This involves a larger size of code, but the time complexity is generally lesser than it is for recursion.

2. **Usage**

Usage of either of these techniques is a trade-off between time complexity and size of code. If time complexity is the point of focus, and the number of recursive calls would be large, it is better to use iteration. However, if time complexity is not an issue and shortness of code is, recursion would be the way to go.

3. Overhead

Recursion has a large amount of Overhead as compared to Iteration.

•**Recursion**: Recursion has the overhead of repeated function calls, that is due to the repetitive calling of the same function, the time complexity of the code increases many fold.

•**Iteration**: Iteration does not involve any such overhead.

4. Infinite Repetition

- Stacks are aalso used in following
  - compilers
    - parsing data between delimiters (brackets)
  - operating systems
    - program stack
  - virtual machines
    - manipulating numbers
      - pop 2 numbers off stack, do work (such as add)
      - push result back on stack and repeat
  - artificial intelligence
    - finding a path

JECRC UNIVERSITY
BUILD YOUR WORLD

Thanks